Name: KANISHKA SHARMA
ROLL NO: 26
Sec: B

Unit I Assignment: Full Stack Architecture & Planning

Q1. Write a folder structure (as a tree) for a full stack e-commerce app using React (frontend), Express (backend), and MySQL (database). In 2 lines, justify why the routes/ and models/ folders should never import each other directly.

Answer:

```
E-Commerce App
├── Client (React)
│   ├── Components
│   ├── Pages
│   └── Services
├── Server (Express)
│   ├── Routes
│   ├── Models
│   ├── Controllers
│   └── Middleware
└── Database (MySQL)
```

The routes/ folder handles HTTP requests, while the models/ folder handles database structures and operations. Keeping them separate reduces coupling and makes the application easier to maintain.

Q2. This React component fetches data using a hardcoded localhost URL. Explain in 2 lines why this call will fail once deployed to production, and rewrite the fetch line using an environment variable.

Answer:

localhost refers to the local computer where the application is running. After deployment, it will refer to the user's computer instead of the production backend server.

```
fetch(`${process.env.REACT_APP_API_URL}/api/products`)
```

Q3. Write a .env file and the corresponding dotenv-loading code for a Node.js/Express backend that needs PORT, DB_URL, and JWT_SECRET. In 1 line, explain why .env should never be committed to Git.

Answer:

PORT=5000
DB_URL=MySQL database URL
JWT_SECRET=Secret key

The .env file should not be committed to Git because it may contain sensitive information such as database credentials and secret keys.

Q4. A React frontend calling an Express API gets a CORS error in the browser console on every request. Write the exact Express middleware/package code that fixes it, and explain in 1 line why the browser blocks the request in the first place.

Answer:
const cors = require('cors'); app.use(cors());

The browser blocks cross-origin requests for security reasons when the server does not provide the required CORS permission.

Q5. Design a REST API contract (as a table: Endpoint | Method | Request Body | Response) for a "Book Library" system with 4 endpoints covering create, read, update, and delete.

Answer:

Endpoint	Method	Request Body	Response
/api/books	POST	Book details	Created book
/api/books/:id	GET	None	Book details

/api/books/:id	PUT	Updated book details	Updated book
/api/books/:id	DELETE	None	Delete confirmation

These four operations represent CRUD: Create, Read, Update, and Delete.

Q6. This Express code creates a new database connection inside every route handler. Rewrite it so the connection is created once and reused across routes, and explain in 2 lines why the original version is a performance problem.

Answer:

The database connection should be created once and reused by different routes.

```
const db = mysql.createConnection({
host: 'localhost',    user: 'root'
});

app.get('/products', (req, res) => {    db.query('SELECT
* FROM products', (err, rows) => {       res.json(rows);
   });
});
```

Creating a new connection for every request consumes extra time and system resources. Reusing a connection improves performance and efficiency.

Q7. Write the "scripts" section of package.json for a full stack project where npm run dev should start both the backend (nodemon server.js) and the frontend (npm start inside a client folder) together, using the concurrently package.

Answer:

```
"scripts": {
  "backend": "nodemon server.js",
  "frontend": "npm start --prefix client",
```

```
  "dev": "concurrently \"npm run backend\" \"npm run frontend\""
}
```

The concurrently package allows the frontend and backend processes to run at the same time.

Q8. A teammate's code hardcodes an API key directly inside a React component. Rewrite the code using an environment variable instead, and explain the security risk in 2 lines.

Answer:

```
const response = await fetch(
  `https://api.weather.com/data?key=${process.env.REACT_APP_WEATHER_API_KEY}`
);
```

Hardcoding an API key can expose it to users through the frontend source or built application. Environment variables keep the key out of the source code, although sensitive keys should preferably be handled on the backend.

Q9. Write pseudocode (as commented steps) tracing the full request-response cycle when a user submits a login form in a full stack app — from the button click to the database query to the UI update.

Answer:

```
// 1. User enters username and password.

// 2. User clicks the Login button.

// 3. React captures the form submission.

// 4. React sends the login request to the Express backend.

// 5. Express receives the request.

// 6. Backend validates the input.
```

// 7. Backend queries the database for the user.

// 8. Database returns the user record.

// 9. Backend verifies the password.

// 10. Server sends a response to React.

// 11. React receives the response.

// 12. React updates the user interface.

// 13. User gets access to the application.

Q10. This Express server crashes on the very first POST request. Identify the missing line that causes `req.body` to be undefined, and provide the corrected code.

Answer:

The missing line is:

```
app.use(express.json());
```

Corrected code:

```
const express = require('express'); const
app = express();
app.use(express.json());

app.post('/login', (req, res) => {     const {
username, password } = req.body;
res.send(`Welcome ${username}`);
});

app.listen(3000);
```

`express.json()` **parses incoming JSON request data and makes it available through** `req.body`**.**